



Twitter releases new API

Twitter launched a new API, and one of the areas of focus is allowing developers to make better bots. <http://dgit.in/sep20-31>



GitHub backup

Perhaps of the way 2020 is turning out, GitHub has taken the precaution of backing up code in the Arctic. <http://dgit.in/sep20-32>

we can use the -p and -t flags. -t is often mistaken for time period but that's actually indicated by the --duration argument.

```
# Stat process 11904 and 11905.
$simpleperf stat -p 11904,11905 --duration 10

# Stat thread 11904 and 11905.
$simpleperf stat -t 11904,11905 --duration 10
```

Then you have the record command which allows you to store the metrics in a file called perf.data.

```
# Record on process 7394 for 10 seconds, using default event (cpu-cycles), # using default sample frequency (4000 samples per second), writing records # to perf.data.
$simpleperf record -p 7394 --duration 10
simpleperf I 07-11 21:44:11 17522 17522 cmd_record.
cpp:316] Samples recorded: 21430. Samples lost: 0.
```

You can also decide whether you want to continuously record or pick a certain frequency to record the process. We won't be getting into that right now. Instead, let's access the report that was stored during the recording phase.

```
# Reports perf.data, using only records sampled in libsudo-game-jni.so, # grouping records using thread name(comm), process id(pid), thread id(tid), # function name(symbol), and showing sample count for each row.
$simpleperf report --dsos /data/app/com.example.sudo-game-2/lib/arm64/lib-sudo-game-jni.so --sort comm,pid,tid,symbol -n
Cmdline: /data/data/com.example.sudogame/simpleperf
record -p 7394 --duration 10
```

```
Arch: arm64
Event: cpu-cycles (type 0, config 0)
Samples: 28235
Event count: 546356211

Overhead Sample Command
Pid Tid Symbol
59.25% 16680 sudogame
7394 7394 checkValid(Board const&, int, int)
20.42% 5620 su-
dogame 7394 7394
canFindSolution_r(Board&, int, int)
```



```
13.82% 4088 su-
dogame 7394 7394
randomBlock_r(Board&, int, int, int, int)
6.24% 1756 sudogame
7394 7394 @plt
```

The above report was for a process that was monitored to understand how many CPU cycles were consumed by each method. The Overhead column at the very beginning tells us how much of the CPU time was consumed by each of the functions that were running.

ONCE AGAIN IN GRAPHICAL FORMAT

Another tool that's also used for profiling the CPU is the CPU Profiler which is present in the Android Studio. This one is a more visual tool and would be easier to grasp

for folks who aren't used to using a CLI. The tool is found in View > Tool Windows > Profiler, and then you need to select a deployment target device in which your app is supposed to run. You should have the device connected to your computer with USB Debugging enabled in the developer options. This is what you'll end up with.

At the top, you have the event timeline which shows the different app states as and when they occur. This is followed by the CPU timeline which shows how much CPU

compute time is used at any given instance. And the last one is the thread activity timeline which lists each thread pertaining to your app and shows their activity.

Like simpleperf, you can record traces of threads or processors for a certain amount of time in the CPU profiler tool.

TIP OF THE ICEBERG

No one tool can handle every single use case or help you troubleshoot your software. Like Android Studio, a lot of IDEs already have a profiler tool which is pretty much what you'll need. However, when your software has to interact with a lot more programs, then you'll need more specialised tools suited for the job. We've covered a lot of these in our August 2020 FastTrack, hopefully, that'll be of some help.